# Retail Management System

Python Object-Oriented Programming Project

| | |
|---|---|
| **Course** | CECS218 |
| **Institution** | University of Jeddah |
| **Contributors** | Reem Sarhan and Malak Yamani |
| **Technology** | Python |
| **Project type** | Console-based academic system |

## Project Overview

A Python-based retail management application designed to manage customers, memberships, products, bundles, inventory, and orders. The system demonstrates core OOP principles, robust file parsing, custom exceptions, and business-rule validation in an interactive console workflow.

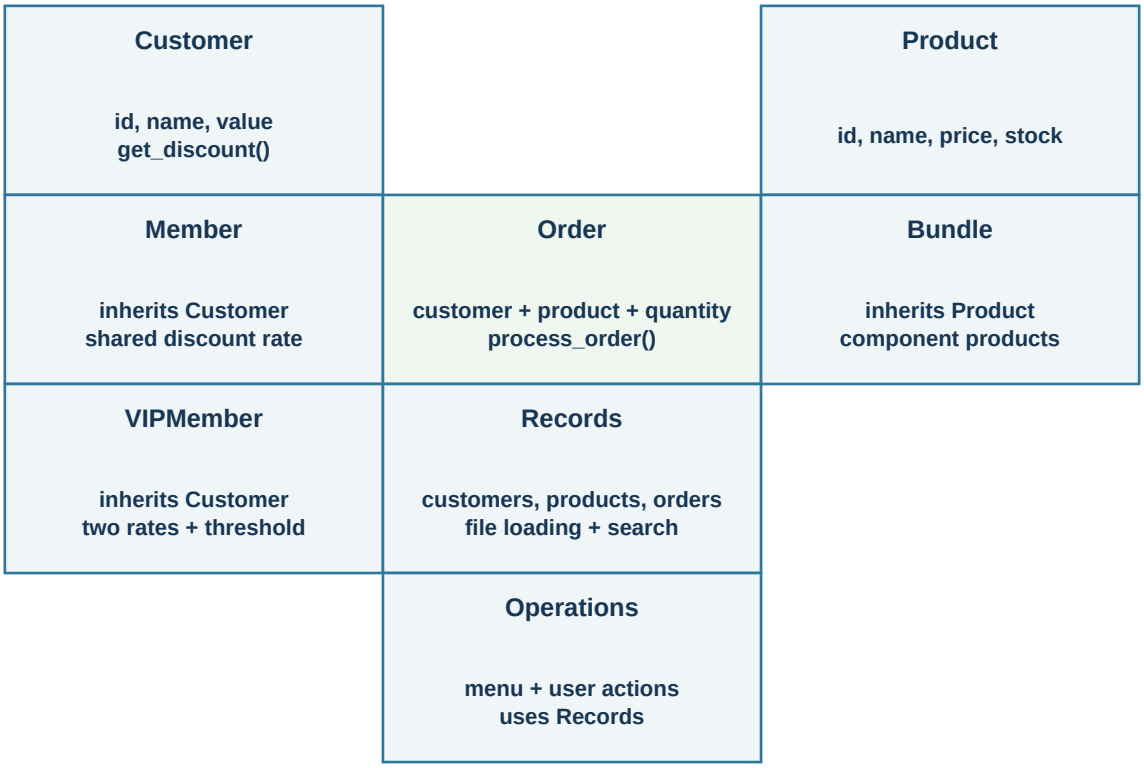
### Key Capabilities

- Customer, Member, and VIP Member account types with different discount behavior.
- Product and bundle management, including bundle pricing at 80% of component totals.
- Order processing with stock updates, spending tracking, discount calculation, and receipt output.
- Data loading from text files with validation for IDs, names, rates, stock, quantities, and dates.
- Administrative controls for VIP rates, VIP threshold, and product price updates.

**Portfolio focus:** Python, OOP, inheritance, polymorphism, file handling, exception handling, data validation, inventory management, and order processing.

# System Architecture

The design separates business entities from record storage and user-facing operations. The relationships below reflect the structure implemented in the source code.




## How the pieces work together

- **Inheritance:** Member and VIPMember extend Customer; Bundle extends Product.
- **Polymorphism:** Customer types implement different discount behavior through get_discount().
- **Composition:** An Order connects one Customer with one Product and a quantity.
- **Persistence:** Records loads and validates customers, products, bundles, and previous orders from text files.
- **Application layer:** Operations provides the interactive menu and coordinates user actions.

# Implementation Highlights

## Validation and reliability

Custom exceptions handle invalid products, quantities, membership inputs, rates, file formats, and names. The program also validates stock availability, positive quantities, duplicate IDs and names, and date parsing.

## Membership logic

Normal customers receive no discount, Members use a shared discount rate, and VIP Members use two discount rates based on a configurable spending threshold. A VIP membership fee is included when a new VIP account is created.

## Inventory and orders

Order processing calculates the total, applies the correct discount, updates stock, tracks customer spending, and prints a receipt. The system can also list all orders or orders for a specific customer.

## Products and bundles

Products store price and stock. Bundles contain multiple products and are automatically priced at 80% of the combined component price.

## File-driven data

The program reads customers.txt, products.txt, and orders.txt rather than hardcoding records, making the system easier to test and extend.

## Recommended GitHub Repository

| retail-management-system/ | |
|---|---|
| `retail_management_system.py` | Main Python application |
| `README.md` | Project overview, features, setup, and file formats |
| `customers.txt` | Required runtime data - add your real project file |
| `products.txt` | Required runtime data - add your real project file |
| `orders.txt` | Previous orders data - add your real project file if used |
| `docs/` | Optional: add this PDF as portfolio documentation |

## LinkedIn Project Description

Developed a Python-based Retail Management System as an academic project, managing customers, products, orders, memberships, and inventory. Applied object-oriented programming concepts, file handling, custom exceptions, data validation, discount calculation, and order processing.

## Top Skills

Python (Programming Language) - Object-Oriented Programming (OOP) - File Handling - Exception Handling - Data Validation